```
"use strict";
/* ================ CONFIGURAÇÃO ================ */
const SUPABASE_URL = "https://aqmnkfbltjyjiuxikjlp.supabase.co";
const SUPABASE_ANON = ["eyJhbGciOiJI","UzI1NiIsInR5","cCI6IkpXVCJ9",".eyJpc3MiOiJ
const LOGO = window.SCRG_LOGO || "";
const APPLE_ICON = window.SCRG_APPLE || "";
try { document.getElementById("ic-fav").href = LOGO; document.getElementById("ic-
const sb = supabase.createClient(SUPABASE_URL, SUPABASE_ANON, { auth: { persistSe

/* ================ UTILITÁRIOS ================ */
const $ = (s, el = document) => el.querySelector(s);
const esc = (s) => String(s ?? "").replace(/[&<>"']/g, c => ({ "&": "&amp;", "<":
const cents = (x) => Math.round(Number(x || 0) * 100);
const eurFmt = new Intl.NumberFormat("pt-PT", { style: "currency", currency: "EUR
const eur = (c) => eurFmt.format((c || 0) / 100);
const numFmt = new Intl.NumberFormat("pt-PT", { maximumFractionDigits: 2 });
const pad = (n) => String(n).padStart(2, "0");
const todayStr = () => { const d = new Date(); return `${d.getFullYear()}-${pad(d
const parseD = (s) => { const [y, m, d] = s.split("-").map(Number); return new Da
const fmtD = (d) => `${d.getUTCFullYear()}-${pad(d.getUTCMonth() + 1)}-${pad(d.ge
const weekStart = (s) => { const d = parseD(s); const wd = (d.getUTCDay() + 6) %
const addDays = (s, n) => { const d = parseD(s); d.setUTCDate(d.getUTCDate() + n)
const MESES = ["jan", "fev", "mar", "abr", "mai", "jun", "jul", "ago", "set", "ou
const MESES_L = ["janeiro", "fevereiro", "março", "abril", "maio", "junho", "julh
const dShort = (s) => { if (!s) return "sem data"; const d = parseD(s); return `$
const dLong = (s) => { if (!s) return "sem data"; const d = parseD(s); return `${
const dtLong = (iso) => { if (!iso) return "—"; const d = new Date(iso); return d
const weekLabel = (ws) => `${dShort(ws)}-${dShort(addDays(ws, 6))}`;
const monthKey = (s) => s.slice(0, 7);
const monthLabel = (k) => { const [y, m] = k.split("-").map(Number); return `${ME
const initials = (n) => n.replace(/["“”]/g, "").replace(/^(T|TA)\s*[--]\s*/, "").
const uuid = () => (crypto.randomUUID ? crypto.randomUUID() : "xxxxxxxx-xxxx-4xxx
function toast(msg, action) {
  const t = $("#toast"); t.innerHTML = ""; t.append(document.createTextNode(msg))
  if (action) { const b = document.createElement("button"); b.className = "toast-
  t.hidden = false; clearTimeout(toast._t); toast._t = setTimeout(() => t.hidden
}
function friendlyError(e) {
  const m = (e && (e.message || e.error_description || e.error)) || String(e);
  if (/PAGAMENTO_SUPERIOR_A_DIVIDA/.test(m)) return m.replace("PAGAMENTO_SUPERIOR
  if (/row-level security|permission denied/i.test(m)) return "Não tens permissão
  if (/Invalid login credentials/i.test(m)) return "Email ou palavra-passe incorr
  if (/duplicate key.*client_ref/i.test(m)) return "Este registo já tinha sido gu
  if (/Failed to fetch|NetworkError/i.test(m)) return "Sem ligação à internet. Te
```

```
  return m;
}

/* ================ ESTADO ================ */
const S = {
  session: null, me: null, isCap: false,
  members: [], types: [], fines: [], payments: [], notes: [],
  view: "inicio", memberId: null,
  rankPeriod: "semana", statPeriod: "epoca", statMember: "", statCat: "",
  finesFilter: { member: "", cat: "", period: "epoca", showVoid: false },
  live: false, charts: [],
};
window.addEventListener("online", () => { if (S.me) { S.offline = false; schedule
window.addEventListener("offline", () => { if (S.me) { S.offline = true; render()
const memberById = (id) => (S.allMembers || S.members).find(m => m.id === id);
const typeById = (id) => S.types.find(t => t.id === id);
const nameByUser = (uid, none = "Sistema") => { if (!uid) return none; const m =
const validFines = () => S.fines.filter(f => f.status === "valida");
const validPays = () => S.payments.filter(p => p.status === "confirmado");

/* ================ DADOS ================ */
const MEMBER_COLS = "id,name,full_name,kind,is_captain,user_id,active,sort_order,
const CACHE_KEY = "scrg-cache-v1";
function saveCache() { try { localStorage.setItem(CACHE_KEY, JSON.stringify({ at:
function readCache(uid) { try { const c = JSON.parse(localStorage.getItem(CACHE_K
function clearCache() { try { localStorage.removeItem(CACHE_KEY); } catch {} }
function online() { return navigator.onLine !== false && !S.offline; }
function guardOnline() { if (!navigator.onLine) { toast("Sem ligação à internet.
async function fetchAll(table, cols, order) {
  const out = []; let from = 0; const step = 1000;
  for (;;) {
    let q = sb.from(table).select(cols).range(from, from + step - 1);
    if (order) q = q.order(order.col, { ascending: order.asc });
    const { data, error } = await q; if (error) throw error;
    out.push(...data); if (data.length < step) break; from += step;
  }
  return out;
}
async function loadAll() {
  const [members, types, fines, payments, notes] = await Promise.all([
    fetchAll("scrg_members", MEMBER_COLS, { col: "sort_order", asc: true }),
    fetchAll("scrg_fine_types", "*", { col: "sort_order", asc: true }),
    fetchAll("scrg_fines", "*", { col: "created_at", asc: false }),
    fetchAll("scrg_payments", "*", { col: "created_at", asc: false }),
    fetchAll("scrg_notes", "*", null),
  ]);
  // atletas removidas da equipa: saem de todas as contas e listas; ficam só no h
```

```
  const removedIds = new Set(members.filter(m => m.removed_at).map(m => m.id));
  Object.assign(S, {
    allMembers: members, members: members.filter(m => !m.removed_at), removed: me
    archive: { fines: fines.filter(f => removedIds.has(f.member_id)), payments: p
    types, fines: fines.filter(f => !removedIds.has(f.member_id)), payments: paym
    notes, offline: false, cachedAt: null });
  saveCache();
}
let reloadT = null;
function scheduleReload() { clearTimeout(reloadT); reloadT = setTimeout(async ()
function subscribeRealtime() {
  const ch = sb.channel("scrg-live");
  ["scrg_fines", "scrg_payments", "scrg_fine_types", "scrg_members"].forEach(t =>
    ch.on("postgres_changes", { event: "*", schema: "public", table: t }, schedul
  ch.subscribe((status) => { S.live = status === "SUBSCRIBED"; const el = $("#liv
  document.addEventListener("visibilitychange", () => { if (!document.hidden) sch
}

/* ================ CÁLCULOS ================ */
function balances() {
  const map = new Map(S.members.map(m => [m.id, { m, fines: 0, count: 0, paid: 0
  for (const f of validFines()) { const b = map.get(f.member_id); if (b) { b.fine
  for (const p of validPays()) { const b = map.get(p.member_id); if (b) b.paid +=
  return [...map.values()].map(b => ({ ...b, debt: b.fines - b.paid }))
    .filter(b => b.m.active || b.fines || b.paid);
}
function teamTotals() {
  const fines = validFines().reduce((s, f) => s + cents(f.total), 0);
  const paid = validPays().reduce((s, p) => s + cents(p.amount), 0);
  const count = validFines().reduce((s, f) => s + Number(f.quantity), 0);
  const ws = weekStart(todayStr()), we = addDays(ws, 6);
  const inW = (d) => d && d >= ws && d <= we;
  const wFines = validFines().filter(f => inW(f.occurrence_date)).reduce((s, f) =
  const wPaid = validPays().filter(p => inW(p.payment_date)).reduce((s, p) => s +
  const bs = balances();
  const debtPos = bs.reduce((s, b) => s + Math.max(b.debt, 0), 0);
  const credit = bs.reduce((s, b) => s + Math.max(-b.debt, 0), 0);
  // "debt" = saldo líquido (multas − recebido), regra original; "debtPos" = dinh
  return { fines, paid, debt: fines - paid, count, wFines, wPaid, debtPos, credit
}
// período: {from,to,label,includeUndated}
function periodRange(key) {
  const t = todayStr();
  if (key === "semana") { const ws = weekStart(t); return { from: ws, to: addDays
  if (key === "mes") { const k = monthKey(t); return monthRange(k); }
  if (key && key.startsWith("m:")) return monthRange(key.slice(2));
  if (key && key.startsWith("w:")) { const ws = key.slice(2); return { from: ws,
```

```
  return { from: null, to: null, label: "Época completa", includeUndated: true };
}
function monthRange(k) { const [y, m] = k.split("-").map(Number); const last = ne
const inRange = (d, r) => { if (!r.from) return true; if (!d) return false; retur
const fineInRange = (f, r) => (r.from ? inRange(f.occurrence_date, r) : true);

function weeklySeries(memberId, cat) {
  const fines = validFines().filter(f => (!memberId || f.member_id === memberId)
  const pays = validPays().filter(p => !memberId || p.member_id === memberId);
  const dated = [...fines.map(f => f.occurrence_date), ...pays.map(p => p.payment
  const undatedFines = fines.filter(f => !f.occurrence_date).reduce((s, f) => s +
  if (!dated.length) dated.push(todayStr());
  let start = weekStart(dated.reduce((a, b) => a < b ? a : b));
  const end = weekStart([...dated, todayStr()].reduce((a, b) => a > b ? a : b));
  const weeks = []; for (let w = start; w <= end; w = addDays(w, 7)) weeks.push(w
  const fBy = new Map(weeks.map(w => [w, 0])), pBy = new Map(weeks.map(w => [w, 0
  fines.forEach(f => { if (f.occurrence_date) { const w = weekStart(f.occurrence_
  pays.forEach(p => { const w = weekStart(p.payment_date); pBy.set(w, (pBy.get(w)
  let cum = undatedFines; const debt = [];
  weeks.forEach(w => { cum += (fBy.get(w) || 0) - (pBy.get(w) || 0); debt.push(cu
  return { weeks, fines: weeks.map(w => fBy.get(w) || 0), pays: weeks.map(w => pB
}

function weekSummary(ws) {
  const we = addDays(ws, 6);
  const f = validFines().filter(x => x.occurrence_date && x.occurrence_date >= ws
  const p = validPays().filter(x => x.payment_date >= ws && x.payment_date <= we)
  const per = {}; f.forEach(x => per[x.member_id] = (per[x.member_id] || 0) + cen
  const topE = Object.entries(per).sort((a, b) => b[1] - a[1])[0];
  const ser = weeklySeries(); const past = ser.weeks.map((w, i) => [w, ser.fines[
  const avg = past.length ? Math.round(past.reduce((s, x) => s + x[1], 0) / past.
  return { fines: f.reduce((s, x) => s + cents(x.total), 0), count: f.reduce((s,
    top: topE ? { name: memberById(topE[0])?.name, v: topE[1] } : null, avg };
}
function activityItems() {
  const items = [];
  S.fines.forEach(f => { items.push({ at: f.created_at, kind: "f", obj: f, text:
    if (f.status === "anulada" && f.updated_at) items.push({ at: f.updated_at, ki
  S.payments.forEach(p => { items.push({ at: p.created_at, kind: "p", obj: p, tex
    if (p.status === "anulado" && p.updated_at) items.push({ at: p.updated_at, ki
  return items.sort((a, b) => b.at.localeCompare(a.at));
}
function openActivity() {
  const items = activityItems().slice(0, 200);
  openSheet("Atividade da equipa", `<p class="hint" style="margin-top:0">Tudo o q
    <div class="list">${items.map(it => `<div class="li tap" ${it.kind[0] === "f"
      <div class="main"><div class="t">${esc(it.text)}</div><div class="d">${dtLo
```

```
      <div class="amt" style="color:${it.kind === "p" ? "var(--good)" : it.kind.e
    root.querySelectorAll("[data-afine]").forEach(el => el.onclick = () => openFi
    root.querySelectorAll("[data-apay]").forEach(el => el.onclick = () => openPay
  });
}

/* ================ GRÁFICOS ================ */
const cssVar = (n) => getComputedStyle(document.documentElement).getPropertyValue
function destroyCharts() { S.charts.forEach(c => c.destroy()); S.charts = []; }
function baseOpts(extra = {}) {
  const ink2 = cssVar("--ink-2"), grid = cssVar("--chart-grid");
  return Object.assign({
    responsive: true, maintainAspectRatio: false, animation: { duration: 300 },
    plugins: { legend: { labels: { color: ink2, boxWidth: 12, font: { family: "Ar
      tooltip: { callbacks: { label: (c) => `${c.dataset.label}: ${eur(c.raw)}` }
    scales: {
      x: { ticks: { color: ink2, maxRotation: 0, autoSkip: true, font: { size: 11
      y: { beginAtZero: true, ticks: { color: ink2, callback: (v) => eur(v).repla
  }, extra);
}
function chart(id, cfg) { const el = document.getElementById(id); if (!el) return
function weeklyChart(id, ser, opts = {}) {
  const ds = [];
  if (opts.fines !== false) ds.push({ type: "bar", label: "Multas", data: ser.fin
  if (opts.pays !== false) ds.push({ type: "bar", label: "Pagamentos", data: ser.
  chart(id, { type: "bar", data: { labels: ser.weeks.map(w => dShort(w)), dataset
}
let mqTheme = matchMedia("(prefers-color-scheme: dark)");
mqTheme.addEventListener?.("change", () => render());

/* ================ ÍCONES ================ */
const IC = {
  inicio: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-widt
  atletas: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wid
  multas: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-widt
  pagamentos: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
  estatisticas: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strok
  rankings: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-wi
  admin: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width
};
const NAV = [["inicio", "Início"], ["atletas", "Atletas"], ["multas", "Multas"],

/* ================ RENDER PRINCIPAL ================ */
function go(view, memberId = null) { S.view = view; S.memberId = memberId; locati
window.addEventListener("hashchange", () => {
  const h = location.hash.slice(1);
  if (h.startsWith("atleta-")) { S.view = "perfil"; S.memberId = h.slice(7); }
```

```
  else if (NAV.some(n => n[0] === h)) { S.view = h; S.memberId = null; }
  if (S.me) render();
});
function shell(content) {
  const nav = NAV.filter(n => n[0] !== "admin" || S.isCap).map(([k, l, sh]) =>
    `<button data-go="${k}" aria-label="${l}" aria-current="${(S.view === k || (S
  return `
  <header class="top"><div class="top-in">
    <img src="${LOGO}" alt="Emblema SCRG">
    <div class="brand"><h1>MULTAS SCRG 26/27</h1>
      <small><span class="live ${S.live ? "" : "off"}" id="live"><i></i></span> $
    <button class="icon-btn" id="logout">Sair</button>
  </div></header>
  <main id="main">${(S.offline || !navigator.onLine) ? `<div class="offline">Sem
  <nav class="tabs" aria-label="Menu"><div class="in">${nav}</div></nav>`;
}
function render() {
  destroyCharts();
  if (!S.me) return;
  if (S.view === "admin" && !S.isCap) S.view = "inicio";
  const views = { inicio: vInicio, atletas: vAtletas, perfil: vPerfil, multas: vM
  const v = (views[S.view] || vInicio)();
  $("#app").innerHTML = shell(v.html);
  $("#logout").onclick = async () => { clearCache(); await sb.auth.signOut(); loc
  document.querySelectorAll("[data-go]").forEach(b => b.onclick = () => go(b.data
  document.querySelectorAll("[data-member]").forEach(el => el.onclick = () => go(
  document.querySelectorAll("[data-fine]").forEach(el => el.onclick = () => openF
  document.querySelectorAll("[data-pay]").forEach(el => el.onclick = () => openPa
  v.after && v.after();
  if (S.offline || !navigator.onLine) document.querySelectorAll("#q-fine,#q-pay,#
}

/* ---------- blocos reutilizáveis ---------- */
function fineRow(f, showMember = true) {
  const t = typeById(f.fine_type_id), m = memberById(f.member_id);
  const q = Number(f.quantity) !== 1 ? `${numFmt.format(f.quantity)} × ` : "";
  const tag = f.status === "anulada" ? '<span class="pill void">anulada</span>' :
  return `<div class="li tap" data-fine="${f.id}">
    ${showMember ? `<div class="av ${m?.is_captain ? "cap" : ""}">${esc(initials(
    <div class="main"><div class="t">${showMember ? esc(m?.name) + " · " : ""}${e
      <div class="d">${q}${eur(cents(f.unit_price))} · ${f.occurrence_date ? dSho
    <div class="amt" style="${f.status === "anulada" ? "text-decoration:line-thro
}
function payRow(p, showMember = true) {
  const m = memberById(p.member_id);
  const tag = p.status === "anulado" ? '<span class="pill void">anulado</span>' :
  return `<div class="li tap" data-pay="${p.id}">
```

```
    ${showMember ? `<div class="av">${esc(initials(m?.name || "?"))}</div>` : ""}
    <div class="main"><div class="t">${showMember ? esc(m?.name) : "Pagamento"}</
      <div class="d">${dShort(p.payment_date)}${p.method ? " · " + esc(p.method)
    <div class="amt" style="color:var(--good);${p.status === "anulado" ? "text-de
}
function debtPill(d) { return d > 0 ? `<span class="pill debt">deve ${eur(d)}</sp

/* ================ INÍCIO ================ */
function vInicio() {
  const T = teamTotals();
  const ser = weeklySeries();
  const lastF = S.fines.filter(f => f.status === "valida").slice(0, 5);
  const lastP = S.payments.filter(p => p.status === "confirmado").slice(0, 5);
  const top = balances().sort((a, b) => b.fines - a.fines || a.m.name.localeCompa
  const max = Math.max(1, ...top.map(b => b.fines));
  const ws = weekStart(todayStr());
  const mine = balances().find(b => b.m.id === S.me.id);
  const wk = weekSummary(ws);
  return { html: `
    ${mine ? `<div class="me-card" data-member="${mine.m.id}" role="button" aria-
      <div><div class="lbl">A minha conta · ${esc(mine.m.name)}</div><div class="
        <div class="small muted">Multas ${eur(mine.fines)} · Pagaste ${eur(mine.p
      <span aria-hidden="true" style="font-size:1.8rem;color:var(--ink-2)">›</spa
    <div class="page-h"><div><h2>Balneário</h2><p>Contas da equipa em tempo real
      ${S.isCap ? `<div class="row"><button class="btn primary" id="q-fine">+ Mul
    <div class="grid g2 g3">
      <div class="kpi red"><div class="l">Total de multas</div><div class="v">${e
      <div class="kpi"><div class="l">Total recebido</div><div class="v" style="c
      <div class="kpi black"><div class="l">Total em dívida</div><div class="v">$
      <div class="kpi"><div class="l">N.º de multas</div><div class="v">${numFmt.
      <div class="kpi"><div class="l">Multas desta semana</div><div class="v">${e
      <div class="kpi"><div class="l">Recebido esta semana</div><div class="v">${
    </div>
    <div class="card" style="margin-top:12px"><h3>Resumo da semana <small>${weekL
      <p style="margin:0">${wk.fines ? `${eur(wk.fines)} em multas (${numFmt.form
    <div class="grid md2" style="margin-top:12px">
      <div class="card"><h3>Evolução semanal <small>seg–dom</small></h3>
        <div class="chart-box"><canvas id="ch-home"></canvas></div>
        ${ser.undatedFines ? `<p class="hint">Não inclui ${eur(ser.undatedFines)}
      <div class="card"><h3>Classificação <small>valor em multas</small></h3>
        <div class="list">${top.map((b, i) => `<div class="li tap" data-member="$
          <div class="main"><div class="t">${esc(b.m.name)}</div><div class="bar"
          <div class="amt">${eur(b.fines)}</div></div>`).join("")}</div>
        <button class="btn block ghost sm" data-go="rankings" style="margin-top:8
      <div class="card"><h3>Últimas multas</h3><div class="list">${lastF.map(f =>
      <div class="card"><h3>Últimos pagamentos</h3><div class="list">${lastP.map(
    </div>
```

```
    <button class="btn block" id="activity" style="margin-top:12px">Ver toda a at
    after() {
      weeklyChart("ch-home", ser);
      $("#q-fine") && ($("#q-fine").onclick = () => openFineForm());
      $("#q-pay") && ($("#q-pay").onclick = () => openPayForm());
      $("#activity").onclick = openActivity;
    } };
}

/* ================ ATLETAS ================ */
function vAtletas() {
  const bs = balances();
  const T = teamTotals();
  const rows = bs.map(b => `<tr data-member="${b.m.id}">
    <td><div class="row" style="gap:8px;flex-wrap:nowrap"><b>${esc(b.m.name)}</b>
    <td class="r">${eur(b.fines)}</td><td class="r" style="color:var(--good)">${e
    <td class="r" style="font-weight:700;color:${b.debt > 0 ? "var(--red)" : b.de
  return { html: `
    <div class="page-h"><div><h2>Atletas</h2><p>Toca numa atleta para ver o perfi
    <div class="card"><div class="tbl-wrap"><table>
      <thead><tr><th>Atleta</th><th class="r">Multas (€)</th><th class="r">Pago (
      <tbody>${rows}</tbody>
      <tfoot><tr><td>Equipa</td><td class="r">${eur(T.fines)}</td><td class="r">$
    </table></div></div>` };
}

/* ================ PERFIL ================ */
function vPerfil() {
  const m = memberById(S.memberId);
  if (!m) return { html: `<div class="empty">Atleta não encontrada.</div>` };
  const b = balances().find(x => x.m.id === m.id) || { fines: 0, paid: 0, debt: 0
  const fines = S.fines.filter(f => f.member_id === m.id);
  const pays = S.payments.filter(p => p.member_id === m.id);
  const ser = weeklySeries(m.id);
  const byCat = {};
  validFines().filter(f => f.member_id === m.id).forEach(f => { const t = typeByI
  const cats = Object.entries(byCat).sort((a, b) => b[1] - a[1]);
  const maxC = Math.max(1, ...cats.map(c => c[1]));
  return { html: `
    <button class="btn ghost sm" data-go="atletas">← Atletas</button>
    <div class="page-h" style="margin-top:10px"><div><h2>${esc(m.name)}</h2>
      <p>${m.full_name ? esc(m.full_name) + " · " : ""}${m.kind === "equipa_tecni
      ${debtPill(b.debt)}</div>
    <div class="grid g2 g3">
      <div class="kpi red"><div class="l">Multas</div><div class="v">${eur(b.fine
      <div class="kpi"><div class="l">Pago</div><div class="v" style="color:var(-
      <div class="kpi black"><div class="l">${b.debt < 0 ? "Crédito" : "Em dívida
```

```
    </div>
    ${S.isCap ? `<div class="row" style="margin-top:12px"><button class="btn prim
    <div class="grid md2" style="margin-top:12px">
      <div class="card"><h3>Evolução semanal</h3><div class="chart-box"><canvas i
        ${ser.undatedFines ? `<p class="hint">Mais ${eur(ser.undatedFines)} impor
      <div class="card"><h3>Por tipo de multa</h3>${cats.length ? `<div class="li
      <div class="card"><h3>Multas <small>${fines.length} registos</small></h3><d
      <div class="card"><h3>Pagamentos <small>${pays.length} registos</small></h3
    </div>`,
    after() {
      weeklyChart("ch-p", ser);
      $("#p-fine") && ($("#p-fine").onclick = () => openFineForm([m.id]));
      $("#p-pay") && ($("#p-pay").onclick = () => openPayForm(m.id));
    } };
}

/* ================= MULTAS ================= */
function periodOptions(sel, withPresets = true) {
  const dates = [...S.fines.map(f => f.occurrence_date), ...S.payments.map(p => p
  const months = [...new Set(dates.map(monthKey))].sort().reverse();
  const weeks = [...new Set(dates.map(weekStart))].sort().reverse();
  const o = (v, l) => `<option value="${v}" ${sel === v ? "selected" : ""}>${esc(
  return o("epoca", "Época completa") + (withPresets ? o("semana", "Semana atual"
    (months.length ? `<optgroup label="Meses">${months.map(k => o("m:" + k, month
    (weeks.length ? `<optgroup label="Semanas">${weeks.map(w => o("w:" + w, weekL
}
const memberOptions = (sel, all = "Todas") => `<option value="">${all}</option>`
const catList = () => [...new Set(S.types.map(t => t.category))];
const catOptions = (sel) => `<option value="">Todas</option>` + catList().map(c =

function vMultas() {
  const F = S.finesFilter, r = periodRange(F.period);
  const list = S.fines.filter(f => (F.showVoid || f.status === "valida") && (!F.m
    (!F.cat || typeById(f.fine_type_id)?.category === F.cat) && fineInRange(f, r)
    .sort((a, b) => (b.occurrence_date || "").localeCompare(a.occurrence_date ||
  const tot = list.filter(f => f.status === "valida").reduce((s, f) => s + cents(
  return { html: `
    <div class="page-h"><div><h2>Multas</h2><p>${list.length} registos · ${eur(to
      ${S.isCap ? `<button class="btn primary" id="new-fine">+ Nova multa</button
    <div class="card"><div class="grid g2 g3">
      <label class="f">Atleta<select id="ff-m">${memberOptions(F.member)}</select
      <label class="f">Categoria<select id="ff-c">${catOptions(F.cat)}</select></
      <label class="f">Período<select id="ff-p">${periodOptions(F.period)}</selec
    </div>
    <label class="row small" style="margin-top:10px"><input type="checkbox" id="f
    <div class="card" style="margin-top:12px"><div class="list">${list.map(f => f
      ${r.from ? `<p class="hint">Filtrado pela data da ocorrência. As multas imp
```

```
    after() {
      $("#ff-m").onchange = (e) => { F.member = e.target.value; render(); };
      $("#ff-c").onchange = (e) => { F.cat = e.target.value; render(); };
      $("#ff-p").onchange = (e) => { F.period = e.target.value; render(); };
      $("#ff-v").onchange = (e) => { F.showVoid = e.target.checked; render(); };
      $("#new-fine") && ($("#new-fine").onclick = () => openFineForm());
    } };
}

/* ================ PAGAMENTOS ================ */
function vPagamentos() {
  const list = [...S.payments].sort((a, b) => b.payment_date.localeCompare(a.paym
  const T = teamTotals();
  const debtors = balances().filter(b => b.debt > 0).sort((a, b) => b.debt - a.de
  return { html: `
    <div class="page-h"><div><h2>Pagamentos</h2><p>Caixa: ${eur(T.paid)} recebido
      ${S.isCap ? `<button class="btn dark" id="new-pay">+ Registar pagamento</bu
    <div class="grid md2">
      <div class="card"><h3>Histórico <small>${list.length} registos</small></h3>
      <div class="card"><h3>Por receber</h3><div class="list">${debtors.map(b =>
    </div>`,
    after() { $("#new-pay") && ($("#new-pay").onclick = () => openPayForm()); } }
}

/* ================ ESTATÍSTICAS ================ */
function vStats() {
  const r = periodRange(S.statPeriod);
  const mId = S.statMember || null, cat = S.statCat || null;
  const ser = weeklySeries(mId, cat);
  // limitar semanas ao período
  const idx = ser.weeks.map((w, i) => i).filter(i => !r.from || (addDays(ser.week
  const pick = (arr) => idx.map(i => arr[i]);
  const s2 = { weeks: pick(ser.weeks), fines: pick(ser.fines), pays: pick(ser.pay
  const fines = validFines().filter(f => (!mId || f.member_id === mId) && (!cat |
  const pays = validPays().filter(p => (!mId || p.member_id === mId) && inRange(p
  const fT = fines.reduce((s, f) => s + cents(f.total), 0), pT = pays.reduce((s,
  const per = {}; fines.forEach(f => per[f.member_id] = (per[f.member_id] || 0) +
  const comp = S.members.filter(m => m.active || per[m.id]).map(m => [m.name, per
  const byCat = {}; fines.forEach(f => { const c = typeById(f.fine_type_id)?.cate
  const undated = fines.filter(f => !f.occurrence_date).reduce((s, f) => s + cent
  const curW = weekStart(todayStr()); const ci = ser.weeks.indexOf(curW);
  const thisW = ci >= 0 ? ser.fines[ci] : 0; const prevW = ser.weeks.map((w, i) =
  const avgW = prevW.length ? Math.round(prevW.reduce((a, x) => a + x[1], 0) / pr
  const byType = {}; fines.forEach(f => { const k = f.fine_type_id; byType[k] = b
  const topTypes = Object.entries(byType).sort((a, b) => b[1].n - a[1].n || b[1].
  const cats = catList(); const catWeeks = s2.weeks;
  const catData = cats.map(c => catWeeks.map(w => fines.filter(f => f.occurrence_
```

```
  return { html: `
    <div class="page-h"><div><h2>Estatísticas</h2><p>${esc(r.label)}${mId ? " · "
    <div class="card"><div class="grid g2 g3">
      <label class="f">Período<select id="sf-p">${periodOptions(S.statPeriod)}</s
      <label class="f">Atleta<select id="sf-m">${memberOptions(S.statMember)}</se
      <label class="f">Categoria de multa<select id="sf-c">${catOptions(S.statCat
    </div><p class="hint" style="margin-top:8px">Multas contam pela data da ocorr
    <div class="grid g2" style="margin-top:12px">
      <div class="kpi red"><div class="l">Multas no período</div><div class="v">$
      <div class="kpi"><div class="l">Pagamentos no período</div><div class="v" s
      <div class="kpi black"><div class="l">Esta semana</div><div class="v">${eur
      <div class="kpi"><div class="l">Tipo mais frequente</div><div class="v" sty
    </div>
    <div class="grid md2" style="margin-top:12px">
      <div class="card"><h3>Multas por semana</h3><div class="chart-box"><canvas
      <div class="card"><h3>Pagamentos por semana</h3><div class="chart-box"><can
      <div class="card"><h3>Dívida acumulada</h3><div class="chart-box"><canvas i
        <p class="hint">Começa nos ${eur(ser.undatedFines)} importados do Excel (
      <div class="card"><h3>Comparação entre atletas</h3><div class="chart-box ta
      <div class="card"><h3>Categorias ao longo da época <small>por semana</small
        <p class="hint">Mostra que tipo de multas está a subir ou a descer. Só in
      <div class="card"><h3>Multas mais frequentes <small>ocorrências</small></h3
      <div class="card"><h3>Por categoria</h3><div class="list">${Object.entries(
    </div>`,
    after() {
      $("#sf-p").onchange = (e) => { S.statPeriod = e.target.value; render(); };
      $("#sf-m").onchange = (e) => { S.statMember = e.target.value; render(); };
      $("#sf-c").onchange = (e) => { S.statCat = e.target.value; render(); };
      weeklyChart("ch-f", s2, { pays: false });
      weeklyChart("ch-pay", s2, { fines: false });
      chart("ch-d", { type: "line", data: { labels: s2.weeks.map(dShort), dataset
      const palette = [cssVar("--red"), cssVar("--chart-pay"), cssVar("--warn"),
      chart("ch-cat", { type: "bar", data: { labels: catWeeks.map(dShort), datase
        options: baseOpts({ scales: { x: { stacked: true, ticks: { color: cssVar(
      chart("ch-c", { type: "bar", data: { labels: comp.map(c => c[0]), datasets:
        options: baseOpts({ indexAxis: "y", plugins: { legend: { display: false }
          scales: { x: { beginAtZero: true, ticks: { color: cssVar("--ink-2"), ca
    } };
}

/* ================ RANKINGS ================ */
function vRankings() {
  const key = S.rankPeriod; const r = periodRange(key);
  const fines = validFines().filter(f => fineInRange(f, r));
  const agg = new Map(S.members.filter(m => m.active).map(m => [m.id, { m, v: 0,
  fines.forEach(f => { const a = agg.get(f.member_id); if (a) { a.v += cents(f.to
  const arr = [...agg.values()];
```

```
  const byV = [...arr].filter(a => a.v > 0).sort((a, b) => b.v - a.v || b.n - a.n
  const byN = [...arr].filter(a => a.n > 0).sort((a, b) => b.n - a.n || b.v - a.v
  const byDebt = balances().filter(b => b.debt > 0).sort((a, b) => b.debt - a.deb
  // ranking semanal: líder de cada semana
  const weekly = new Map();
  validFines().filter(f => f.occurrence_date && fineInRange(f, r)).forEach(f => {
  const weekLeaders = [...weekly.entries()].sort((a, b) => b[0].localeCompare(a[0
  const podium = (list, fmt) => {
    if (!list.length) return '<div class="empty">Ninguém no pódio neste período.
    const [a, b, c] = list;
    const p = (x, cls, n) => x ? `<div class="pod ${cls}"><div class="p">${n}</di
    return `<div class="podium">${p(b, "p2", 2)}${p(a, "p1", 1)}${p(c, "p3", 3)}<
      (list.length > 3 ? `<div class="list">${list.slice(3).map((x, i) => `<div c
  };
  const seg = [["semana", "Semana atual"], ["mes", "Mês"], ["epoca", "Época"]].ma
  return { html: `
    <div class="page-h"><div><h2>Rankings</h2><p>${esc(r.label)}</p></div></div>
    <div class="seg" role="group" aria-label="Período">${seg}</div>
    ${key !== "epoca" ? `<p class="hint" style="margin:8px 2px 0">Contam as multa
    <div class="grid md2" style="margin-top:12px">
      <div class="card"><h3>Rainhas da multa <small>maior valor</small></h3>${pod
      <div class="card"><h3>Reincidentes <small>mais multas</small></h3>${podium(
      <div class="card"><h3>Maior saldo por pagar <small>dívida atual</small></h3
        <p class="hint">A dívida é sempre o saldo atual, independentemente do per
      <div class="card"><h3>Ranking semanal <small>líder de cada semana</small></
        <div class="list">${weekLeaders.map(w => `<div class="li"><div class="mai
    </div>`,
    after() { document.querySelectorAll("[data-rp]").forEach(b => b.onclick = ()
}

/* ================ ADMINISTRAÇÃO ================ */
let adminTab = "contas"; let adminCache = { status: [], audit: [], invites: [], r
function vAdmin() {
  const tabs = [["contas", "Contas"], ["tipos", "Tipos de multa"], ["historico",
    .map(([k, l]) => `<button data-at="${k}" aria-pressed="${adminTab === k}">${l
  let body = "";
  if (adminTab === "contas") {
    const st = new Map(adminCache.status.map(s => [s.member_id, s]));
    body = `<div class="card"><h3>Atletas e contas <small>convites individuais</s
      <p class="hint">O convite gera um link pessoal e de uso único. Envia-o só à
      <div class="list">${S.members.map(m => { const s = st.get(m.id);
        const state = m.access_blocked ? '<span class="pill debt">acesso bloquead
        return `<div class="li"><div class="av ${m.is_captain ? "cap" : ""}">${es
          <div class="main"><div class="t">${esc(m.name)} ${m.is_captain ? '<span
          <div class="d">${s?.email ? esc(s.email) : "sem email"} · ${state}${s?.
          <button class="btn sm" data-edit-member="${m.id}">Gerir</button></div>`
      <button class="btn block" id="add-member" style="margin-top:10px">+ Adicion
```

```
      ${S.removed?.length ? `<div class="card" style="margin-top:12px"><h3>Atleta
        <p class="hint">Não contam para nenhum total, gráfico ou ranking e as atl
        <div class="list">${S.removed.map(r => `<div class="li tap" data-removed=
          <div class="d">Removida a ${dtLong(r.removed_at)} por ${esc(nameByUser(
          <div class="amt" style="color:var(--warn)">${eur(cents(r.refund_amount)
      <div class="card" style="margin-top:12px"><h3>Conferência de totais</h3><di
      ${S.notes.map(n => `<div class="card" style="margin-top:12px"><h3>${esc(n.t
  } else if (adminTab === "tipos") {
    body = `<div class="card"><h3>Tipos de multa e preços</h3>
      <p class="hint">Alterar um preço só afeta multas novas. As multas já regist
      ${catList().map(c => `<h4 style="margin:14px 0 4px;font-family:var(--f-disp
      <button class="btn block" id="add-type" style="margin-top:12px">+ Novo tipo
  } else if (adminTab === "historico") {
    body = `<div class="card"><h3>Histórico de alterações <small>últimas 300</sma
  } else {
    body = `<div class="card"><h3>Exportar relatórios</h3>
      <button class="btn primary block" id="exp-xlsx">Livro Excel completo (.xlsx
      <p class="hint">Saldos, multas, pagamentos e resumo semanal em folhas separ
      <div class="stack"><button class="btn block" data-exp="saldos">Saldos por a
      <button class="btn block" data-exp="multas">Todas as multas</button>
      <button class="btn block" data-exp="pagamentos">Todos os pagamentos</button
      <button class="btn block" data-exp="semanal">Resumo semanal</button></div><
  }
  return { html: `<div class="page-h"><div><h2>Administração</h2><p>Só visível pa
    <div class="seg" role="group">${tabs}</div><div style="margin-top:12px">${bod
    after() {
      document.querySelectorAll("[data-at]").forEach(b => b.onclick = () => { adm
      document.querySelectorAll("[data-edit-member]").forEach(b => b.onclick = ()
      document.querySelectorAll("[data-removed]").forEach(b => b.onclick = () =>
      document.querySelectorAll("[data-edit-type]").forEach(b => b.onclick = () =
      document.querySelectorAll("[data-exp]").forEach(b => b.onclick = () => expo
      $("#exp-xlsx") && ($("#exp-xlsx").onclick = exportXlsx);
      $("#add-member") && ($("#add-member").onclick = () => openMemberForm(null))
      $("#add-type") && ($("#add-type").onclick = () => openTypeForm(null));
      if (!vAdmin._loaded) { vAdmin._loaded = true; loadAdminData(); }
      const rc = $("#recon");
      if (rc && adminCache.recon) {
        const local = new Map(balances().map(b => [b.m.id, b]));
        const diffs = adminCache.recon.filter(r => { const b = local.get(r.member
        const dbF = adminCache.recon.reduce((s, r) => s + cents(r.fines_total), 0
        rc.innerHTML = diffs.length ? `<div class="err">Diferenças em ${diffs.len
          : `<div class="okmsg">Totais conferidos: a app e a base de dados coinci
      }
    } };
}
async function loadAdminData() {
  try {
```

```
    const [st, au] = await Promise.all([sb.rpc("scrg_accounts"), sb.from("scrg_au
    if (!st.error) adminCache.status = st.data || [];
    const bv = await sb.from("scrg_balances").select("member_id,fines_total,paid_
    if (!bv.error) adminCache.recon = bv.data || [];
    if (!au.error) adminCache.audit = au.data || [];
    if (S.view === "admin") { const y = window.scrollY; render(); window.scrollTo
  } catch (e) { console.error(e); }
}
function auditRow(a) {
  const T = { scrg_fines: "Multa", scrg_payments: "Pagamento", scrg_fine_types: "
  const d = a.new_data || a.old_data || {};
  let what = "";
  if (a.table_name === "scrg_fines") what = `${memberById(d.member_id)?.name || "
  else if (a.table_name === "scrg_payments") what = `${memberById(d.member_id)?.n
  else if (a.table_name === "scrg_fine_types") what = `${d.name} · ${eur(cents(d.
  else if (a.table_name === "scrg_members") what = d.name || "";
  else if (a.table_name === "scrg_invites") what = `${memberById(d.member_id)?.na
  let act = { INSERT: "criado", UPDATE: "alterado", DELETE: "eliminado", BLOQUEIO
  if (a.action === "REMOCAO") act += ` (devolver ${eur(cents(d.refund))}${d.reaso
  if (a.action === "BLOQUEIO" && d.reason) act += ` (${d.reason})`;
  if (a.action === "UPDATE" && a.old_data && a.new_data) {
    if (a.old_data.status !== a.new_data.status && /anulad/.test(a.new_data.statu
    else { const ch = Object.keys(a.new_data).filter(k => !["updated_at", "update
  }
  return `<div class="li"><div class="main"><div class="t">${T} ${esc(act)}</div>
}

/* ================= EXPORTAÇÃO ================= */
function reportRows(kind) {
  const n = (c) => Math.round(c || 0) / 100;
  const rows = [];
  if (kind === "saldos") { rows.push(["Atleta", "Multas (€)", "Pago (€)", "Em dív
  if (kind === "multas") { rows.push(["Atleta", "Categoria", "Tipo", "Quantidade"
    S.fines.forEach(f => { const t = typeById(f.fine_type_id); rows.push([memberB
  if (kind === "pagamentos") { rows.push(["Atleta", "Valor (€)", "Data do pagamen
    S.payments.forEach(p => rows.push([memberById(p.member_id)?.name, n(cents(p.a
  if (kind === "semanal") { const s = weeklySeries(); rows.push(["Semana (seg)",
  return rows;
}
function loadScript(src) { return new Promise((res, rej) => { const el = document
async function exportXlsx() {
  try {
    if (typeof XLSX === "undefined") { toast("A preparar o Excel…"); await loadSc
    const wb = XLSX.utils.book_new();
    [["saldos", "Saldos"], ["multas", "Multas"], ["pagamentos", "Pagamentos"], ["
      const rows = reportRows(k); const ws = XLSX.utils.aoa_to_sheet(rows); ws["!
    });
```

```
    XLSX.writeFile(wb, `multas-scrg-2627-${todayStr()}.xlsx`);
    toast("Excel exportado.");
  } catch (e) { toast("Não foi possível criar o Excel. Usa a exportação em CSV.")
}
function exportCsv(kind) {
  const rows = reportRows(kind).map(r => r.map(v => typeof v === "number" ? v.toF
  const csv = "\ufeff" + rows.map(r => r.map(v => `"${String(v ?? "").replace(/"/
  const a = document.createElement("a"); a.href = URL.createObjectURL(new Blob([c
  a.download = `multas-scrg-${kind}-${todayStr()}.csv`; document.body.appendChild
  toast("Relatório exportado.");
}

/* ================= FOLHAS (MODAIS) ================= */
function openSheet(title, html, onMount) {
  const root = $("#sheet-root");
  root.innerHTML = `<div class="sheet-bg" id="sbg"><div class="sheet" role="dialo
    <div class="grab"></div><div class="sheet-h"><h3>${esc(title)}</h3><button cl
  $("#sclose").onclick = closeSheet;
  $("#sbg").onclick = (e) => { if (e.target.id === "sbg") closeSheet(); };
  document.body.style.overflow = "hidden";
  onMount && onMount(root);
}
function closeSheet() { $("#sheet-root").innerHTML = ""; document.body.style.over
document.addEventListener("keydown", (e) => { if (e.key === "Escape") closeSheet(

/* ---------- nova multa ---------- */
function openFineForm(preselect = []) {
  if (!S.isCap) return;
  const sel = new Set(preselect); const ref = uuid(); let busy = false;
  const typesOpts = catList().map(c => `<optgroup label="${esc(c)}">${S.types.fil
  const active = S.members.filter(m => m.active);
  openSheet("Nova multa", `
    <form id="fine-f" class="stack" novalidate>
      <div class="f"><span style="font-weight:600;font-size:.86rem">Atletas <span
        <div class="row" style="gap:6px"><button type="button" class="btn sm" id=
        <div class="chips" id="chips">${active.map(m => `<button type="button" cl
      <div class="f"><span style="font-weight:600;font-size:.86rem">Tipo de multa
        <div class="type-chips" id="tchips">${frequentTypes().map(t => `<button t
        <select id="ft" required aria-label="Tipo de multa"><option value="">Outr
      <div class="grid g2">
        <label class="f">Quantidade<div class="stepper"><button type="button" id=
        <label class="f">Data da ocorrência<input id="fd" type="date" value="${to
      </div>
      <label class="f">Observações <span class="hint">opcional</span><textarea id
      <div class="totalbar"><span id="fsum">Escolhe atletas e tipo</span><b id="f
      <div id="ferr" class="err" hidden></div>
      <button class="btn primary block" id="fsave" type="submit">Registar multa</
```

```
    </form>`, () => {
    const upd = () => {
      document.querySelectorAll("[data-chip]").forEach(c => c.setAttribute("aria-
      const t = typeById($("#ft").value); const q = Number($("#fq").value) || 0;
      const unit = t ? cents(t.unit_price) : 0; const each = Math.round(unit * q)
      $("#fsum").textContent = t ? `${sel.size} × ${numFmt.format(q)} × ${eur(uni
      $("#ftot").textContent = eur(each * sel.size);
    };
    document.querySelectorAll("[data-chip]").forEach(c => c.onclick = () => { sel
    $("#sel-ath").onclick = () => { active.filter(m => m.kind === "atleta").forEa
    $("#sel-staff").onclick = () => { active.filter(m => m.kind === "equipa_tecni
    $("#sel-none").onclick = () => { sel.clear(); upd(); };
    $("#qm").onclick = () => { $("#fq").value = Math.max(0.5, (Number($("#fq").va
    $("#qp").onclick = () => { $("#fq").value = (Number($("#fq").value) || 0) + 1
    const syncT = () => document.querySelectorAll("[data-tchip]").forEach(c => c.
    document.querySelectorAll("[data-tchip]").forEach(c => c.onclick = () => { $(
    $("#ft").onchange = () => { syncT(); upd(); }; $("#fq").oninput = upd; upd();
    $("#fine-f").onsubmit = async (e) => {
      e.preventDefault(); if (busy || !guardOnline()) return;
      const err = $("#ferr"); err.hidden = true;
      const t = $("#ft").value, q = Number($("#fq").value), d = $("#fd").value;
      const msg = !sel.size ? "Escolhe pelo menos uma atleta." : !t ? "Escolhe o
      if (msg) { err.textContent = msg; err.hidden = false; return; }
      busy = true; $("#fsave").disabled = true; $("#fsave").textContent = "A guar
      const batch = sel.size > 1 ? ref : null;
      const rows = [...sel].map((mid, i) => ({ member_id: mid, fine_type_id: t, q
        client_ref: i === 0 ? ref : uuid5ish(ref, mid) }));
      const { error } = await sb.from("scrg_fines").insert(rows);
      if (error && !/client_ref/.test(error.message)) { busy = false; $("#fsave")
      closeSheet(); await loadAll(); render();
      const refs = rows.map(r => r.client_ref);
      toast(sel.size > 1 ? `${sel.size} multas registadas.` : "Multa registada.",
        const ids = S.fines.filter(f => refs.includes(f.client_ref) && f.status =
        const { error } = await sb.from("scrg_fines").update({ status: "anulada",
        if (error) { toast(friendlyError(error)); return; }
        await loadAll(); render(); toast("Registo desfeito (fica no histórico com
      } });
    };
  });
}
// tipos mais usados (para escolher com um toque); sem histórico, mostra os de jo
function frequentTypes() {
  const cnt = {}; S.fines.filter(f => f.source === "app").forEach(f => cnt[f.fine
  const used = Object.entries(cnt).sort((a, b) => b[1] - a[1]).map(([id]) => type
  const game = ["Derrota (Jogadoras)", "Empate", "Consumo Mínimo (1€/jogo)", "Atr
  const out = []; [...used, ...game].forEach(t => { if (!out.includes(t)) out.pus
  return out.slice(0, 6);
```

```
}
// referência determinística por atleta dentro do mesmo formulário (evita duplica
function uuid5ish(base, mid) { const h = (base + mid).split("").reduce((a, c) =>

/* ---------- detalhe / editar multa ---------- */
function openFineDetail(id) {
  const f = S.fines.find(x => x.id === id); if (!f) return;
  const t = typeById(f.fine_type_id), m = memberById(f.member_id);
  const can = S.isCap && f.status === "valida";
  openSheet("Multa", `
    <div class="card stack">
      <div class="row"><b class="sp">${esc(m?.name)}</b><span class="amt" style="
      <div>${esc(t?.category)} · ${esc(t?.name)}</div>
      <div class="small muted">${numFmt.format(f.quantity)} × ${eur(cents(f.unit_
      <div class="small"><b>Data da ocorrência:</b> ${f.occurrence_date ? dLong(f
        <b>Registo:</b> ${dtLong(f.created_at)} por ${esc(nameByUser(f.created_by
        ${f.batch_id ? "<br>Registada em grupo com outras atletas" : ""}</div>
      ${f.notes ? `<div class="small"><b>Observações:</b> ${esc(f.notes)}</div>`
      ${f.status === "anulada" ? `<div class="warnbox">Anulada: ${esc(f.void_reas
    </div>
    ${can ? `<form id="fe" class="stack" style="margin-top:14px" novalidate>
      <h4 style="margin:0;font-family:var(--f-display);text-transform:uppercase;f
      <label class="f">Tipo de multa<select id="et">${S.types.map(x => `<option v
        <span class="hint">Se mudares o tipo, o preço passa a ser o atual desse t
      <div class="grid g2"><label class="f">Quantidade<input id="eq" type="number
      <label class="f">Data da ocorrência<input id="ed" type="date" value="${f.oc
      <label class="f">Observações<textarea id="en">${esc(f.notes || "")}</textar
      <div id="eerr" class="err" hidden></div>
      <button class="btn primary block" type="submit">Guardar correção</button>
    </form>
    <div class="stack" style="margin-top:14px">${(() => { const b = balances().fi
      <label class="f">Anular multa — motivo<input id="vr" placeholder="Ex.: regi
      <button class="btn block" id="void" style="color:var(--red)">Anular multa</
    if (!can) return;
    $("#fe").onsubmit = async (e) => {
      e.preventDefault(); if (!guardOnline()) return;
      const q = Number($("#eq").value), d = $("#ed").value || null;
      if (!(q > 0)) { $("#eerr").textContent = "Quantidade inválida."; $("#eerr")
      if (!d && f.source !== "excel_import") { $("#eerr").textContent = "Indica a
      const { error } = await sb.from("scrg_fines").update({ fine_type_id: $("#et
      if (error) { $("#eerr").textContent = friendlyError(error); $("#eerr").hidd
      closeSheet(); toast("Multa corrigida. A alteração ficou no histórico."); aw
    };
    $("#void").onclick = async () => {
      if (!guardOnline()) return;
      const r = $("#vr").value.trim(); if (!r) { $("#vr").focus(); toast("Indica
      const { error } = await sb.from("scrg_fines").update({ status: "anulada", v
```

```
      if (error) { toast(friendlyError(error)); return; }
      closeSheet(); toast("Multa anulada."); await loadAll(); render();
    };
  });
}

/* ---------- novo pagamento ---------- */
function openPayForm(preMember = "", prefill = null) {
  if (!S.isCap) return;
  const ref = uuid(); let busy = false; let needConfirm = false;
  const bs = new Map(balances().map(b => [b.m.id, b]));
  openSheet("Registar pagamento", `
    <form id="pay-f" class="stack" novalidate>
      <label class="f">Atleta<select id="pm" required><option value="">Escolher…<
      <div class="grid g2">
        <label class="f">Valor recebido (€)<input id="pa" type="number" inputmode
        <label class="f">Data do pagamento<input id="pd" type="date" value="${tod
      </div>
      <div class="row" id="quick" style="gap:6px"></div>
      <label class="f">Método <span class="hint">opcional</span><select id="pmeth
      <label class="f">Observação <span class="hint">opcional</span><textarea id=
      <div id="pinfo"></div>
      <div id="perr" class="err" hidden></div>
      <button class="btn dark block" id="psave" type="submit">Registar pagamento<
    </form>`, () => {
    const upd = () => {
      const b = bs.get($("#pm").value); const a = cents($("#pa").value);
      $("#quick").innerHTML = b && b.debt > 0 ? `<button type="button" class="btn
      $("#pall") && ($("#pall").onclick = () => { $("#pa").value = (b.debt / 100)
      needConfirm = !!(b && a > Math.max(b.debt, 0));
      $("#pinfo").innerHTML = !b ? "" : needConfirm ? `<div class="warnbox"><b>O
        <label class="row" style="margin-top:8px;font-weight:700"><input type="ch
        : a ? `<div class="okmsg">Depois deste pagamento: ${b.debt - a === 0 ? "f
    };
    if (prefill) { $("#pa").value = prefill.amount; $("#pd").value = prefill.date
    $("#pm").onchange = upd; $("#pa").oninput = upd; upd();
    $("#pay-f").onsubmit = async (e) => {
      e.preventDefault(); if (busy || !guardOnline()) return;
      const err = $("#perr"); err.hidden = true;
      const m = $("#pm").value, a = cents($("#pa").value), d = $("#pd").value;
      const msg = !m ? "Escolhe a atleta." : !(a > 0) ? "Indica um valor maior qu
      if (msg) { err.textContent = msg; err.hidden = false; return; }
      busy = true; $("#psave").disabled = true; $("#psave").textContent = "A guar
      const { error } = await sb.from("scrg_payments").insert({ member_id: m, amo
      if (error && !/client_ref/.test(error.message)) { busy = false; $("#psave")
      closeSheet(); await loadAll(); render();
      toast("Pagamento registado.", { label: "Desfazer", fn: async () => {
```

```
        const p = S.payments.find(x => x.client_ref === ref && x.status === "conf
        const { error } = await sb.from("scrg_payments").update({ status: "anulad
        if (error) { toast(friendlyError(error)); return; }
        await loadAll(); render(); toast("Pagamento desfeito (fica no histórico c
      } });
    };
  });
}

/* ---------- detalhe / anular pagamento ---------- */
function openPayDetail(id) {
  const p = S.payments.find(x => x.id === id); if (!p) return;
  const m = memberById(p.member_id); const can = S.isCap && p.status === "confirm
  openSheet("Pagamento", `
    <div class="card stack">
      <div class="row"><b class="sp">${esc(m?.name)}</b><span style="font-family:
      <div class="small"><b>Data do pagamento:</b> ${dLong(p.payment_date)}${p.me
        <b>Registo:</b> ${dtLong(p.created_at)} por ${esc(nameByUser(p.created_by
      ${p.credit_confirmed ? `<div class="small"><span class="pill credit">Crédit
      ${p.notes ? `<div class="small"><b>Observação:</b> ${esc(p.notes)}</div>` :
      ${p.status === "anulado" ? `<div class="warnbox">Anulado: ${esc(p.void_reas
    </div>
    ${can ? `<div class="stack" style="margin-top:14px"><p class="hint" style="ma
      <label class="f">Motivo<input id="pvr" placeholder="Ex.: valor errado, era
      <div class="grid g2"><button class="btn primary" id="pfix">Corrigir</button
    if (!can) return;
    const voidIt = async () => {
      if (!guardOnline()) return false;
      const r = $("#pvr").value.trim(); if (!r) { $("#pvr").focus(); toast("Indic
      const { error } = await sb.from("scrg_payments").update({ status: "anulado"
      if (error) { toast(friendlyError(error)); return false; }
      return true;
    };
    $("#pvoid").onclick = async () => { if (!(await voidIt())) return; closeSheet
    $("#pfix").onclick = async () => {
      if (!(await voidIt())) return; await loadAll(); render();
      openPayForm(p.member_id, { amount: Number(p.amount).toFixed(2), date: p.pay
      toast("Pagamento anterior anulado. Regista agora o valor correto.");
    };
  });
}

/* ---------- gerir atleta / convites ---------- */
function blockUi(m) {
  if (m.is_captain || m.user_id === S.me.user_id) return "";
  return `<button class="btn block" id="block1" style="color:var(--red)">Bloquear
    <div id="bconf" class="warnbox stack" hidden><b>Bloquear o acesso de ${esc(m.
```

```
      <span>Deixa de conseguir entrar na app e de ver os dados, a partir de agora
      <label class="f">Motivo <span class="hint">opcional</span><input id="breaso
      <button class="btn primary block" id="block2">Sim, bloquear acesso</button>
}
function removeUi(m) {
  if (m.is_captain || m.user_id === S.me.user_id) return "";
  const b = balances().find(x => x.m.id === m.id) || { fines: 0, paid: 0, debt: 0
  return `<div class="card stack" style="margin-top:14px;border-color:var(--red)"
    <p class="small" style="margin:0">Remove ${esc(m.name)} de todas as contas, l
    <button class="btn block" id="rm1" style="color:var(--red)">Remover da equipa
    <div id="rmconf" class="warnbox stack" hidden>
      <b>Confirmar a remoção de ${esc(m.name)}</b>
      <span>Multas registadas: ${eur(b.fines)} · Pagou: ${eur(b.paid)}</span>
      <span><b>A devolver: ${eur(b.paid)}</b>${b.debt > 0 ? ` · Dívida perdoada:
      <span>O total da equipa passa de ${eur(teamTotals().fines)} para ${eur(team
      <label class="f">Motivo <span class="hint">opcional</span><input id="rmreas
      <label class="row" style="font-weight:700"><input type="checkbox" id="rmok"
      <button class="btn primary block" id="rm2">Remover ${esc(m.name)}</button><
}
function openRemovedDetail(id) {
  const r = memberById(id); if (!r) return;
  const snap = r.removal_snapshot || {};
  const fines = S.archive.fines.filter(f => f.member_id === id), pays = S.archive
  openSheet(r.name + " (removida)", `
    <div class="card stack">
      <div class="small"><b>Removida:</b> ${dtLong(r.removed_at)} por ${esc(nameB
      <div class="grid g2"><div><div class="hint">Multas na altura</div><b>${eur(
        <div><div class="hint">A devolver</div><b style="color:var(--warn)">${eur
    </div>
    <div class="card" style="margin-top:12px"><h3>Multas <small>${fines.length}</
    <div class="card" style="margin-top:12px"><h3>Pagamentos <small>${pays.length
    <div class="stack" style="margin-top:14px"><p class="hint" style="margin:0">R
      <button class="btn block" id="restore">Repor ${esc(r.name)} na equipa</butt
    $("#restore").onclick = async () => {
      if (!guardOnline()) return;
      const btn = $("#restore"); btn.disabled = true;
      const { data, error } = await sb.functions.invoke("scrg-invite", { body: {
      let msg = data?.error; if (error) { try { msg = (await error.context.json()
      btn.disabled = false;
      if (msg) { $("#rres").innerHTML = `<div class="err">${esc(msg)}</div>`; ret
      closeSheet(); toast(`${r.name} voltou à equipa.`); await loadAll(); render(
    };
  });
}
function fineRowStatic(f) {
  const t = typeById(f.fine_type_id);
  return `<div class="li"><div class="main"><div class="t">${esc(t?.name)}</div><
```

```
}
function openMemberForm(id) {
  const m = id ? memberById(id) : null;
  const acc = m ? adminCache.status.find(x => x.member_id === m.id) : null;
  openSheet(m ? m.name : "Adicionar atleta", `
    <form id="mf" class="stack" novalidate>
      <label class="f">Nome a mostrar<input id="mn" value="${esc(m?.name || "")}"
      <label class="f">Nome completo <span class="hint">opcional</span><input id=
      <label class="f">Função<select id="mk"><option value="atleta" ${m?.kind !==
      ${m && !m.is_captain ? `<label class="row"><input type="checkbox" id="ma" $
      <div id="merr" class="err" hidden></div>
      <button class="btn primary block" type="submit">${m ? "Guardar" : "Adiciona
    </form>
    ${m ? `<div class="card stack" style="margin-top:14px"><h3 style="margin:0">A
      ${m.access_blocked ? `<div class="err">Acesso bloqueado${m.blocked_at ? " d
        <button class="btn block" id="unblock">Desbloquear acesso</button>`
      : m.user_id ? `<p class="small" style="margin:0">Conta associada a <b>${esc
        <button class="btn block" id="relink">Gerar novo link de acesso</button>
        <p class="hint" style="margin:0">Útil se a atleta esqueceu a palavra-pass
        ${blockUi(m)}`
      : `<label class="f">Email da atleta<input id="ie" type="email" inputmode="e
        <button class="btn primary block" id="invite">Criar convite individual</b
        <p class="hint" style="margin:0">Cria a conta só para este email, com ace
        ${blockUi(m)}`}
      <div id="ires"></div></div>${removeUi(m)}` : ""}`, () => {
    $("#mf").onsubmit = async (e) => {
      e.preventDefault();
      const row = { name: $("#mn").value.trim(), full_name: $("#mfull").value.tri
      if (!row.name) { $("#merr").textContent = "Indica o nome."; $("#merr").hidd
      if (m && $("#ma")) row.active = $("#ma").checked; else if (!m) row.sort_ord
      const q = m ? sb.from("scrg_members").update(row).eq("id", m.id) : sb.from(
      const { error } = await q;
      if (error) { $("#merr").textContent = /unique|duplicate/i.test(error.messag
      closeSheet(); toast(m ? "Dados guardados." : "Atleta adicionada."); await l
    };
    const showLink = (data) => {
      const link = `${location.origin}${location.pathname}?acesso=${encodeURIComp
      $("#ires").innerHTML = `<div class="okmsg">Link criado para ${esc(data.memb
        <div class="linkbox" id="lnk">${esc(link)}</div><button class="btn block"
      $("#cp").onclick = async () => { try { await navigator.clipboard.writeText(
    };
    const call = async (body, btn) => {
      btn.disabled = true; const old = btn.textContent; btn.textContent = "A cria
      const { data, error } = await sb.functions.invoke("scrg-invite", { body });
      btn.disabled = false; btn.textContent = old;
      let msg = data?.error; if (error) { try { msg = (await error.context.json()
      if (msg) { $("#ires").innerHTML = `<div class="err">${esc(msg)}</div>`; ret
```

```
      showLink(data); loadAll().then(() => loadAdminData());
    };
    $("#invite") && ($("#invite").onclick = () => { const em = $("#ie").value.tri
    $("#relink") && ($("#relink").onclick = () => call({ action: "relink", member
    const doBlock = async (action, btn) => {
      btn.disabled = true;
      const { data, error } = await sb.functions.invoke("scrg-invite", { body: {
      let msg = data?.error; if (error) { try { msg = (await error.context.json()
      btn.disabled = false;
      if (msg) { $("#ires").innerHTML = `<div class="err">${esc(msg)}</div>`; ret
      closeSheet(); toast(action === "block" ? `Acesso de ${m.name} bloqueado.` :
    };
    $("#block1") && ($("#block1").onclick = () => { $("#bconf").hidden = false; $
    $("#block2") && ($("#block2").onclick = () => doBlock("block", $("#block2")))
    $("#unblock") && ($("#unblock").onclick = () => doBlock("unblock", $("#unbloc
    $("#rm1") && ($("#rm1").onclick = () => { $("#rmconf").hidden = false; $("#rm
    $("#rm2") && ($("#rm2").onclick = async () => {
      if (!guardOnline()) return;
      if (!$("#rmok").checked) { toast("Marca a confirmação primeiro."); return;
      const btn = $("#rm2"); btn.disabled = true; btn.textContent = "A remover…";
      const { data, error } = await sb.functions.invoke("scrg-invite", { body: {
      let msg = data?.error; if (error) { try { msg = (await error.context.json()
      if (msg) { btn.disabled = false; btn.textContent = `Remover ${m.name}`; $("
      closeSheet(); toast(`${m.name} removida. Devolver ${eur(cents(data.refund))
    });
  });
}

/* ---------- tipos de multa ---------- */
function openTypeForm(id) {
  const t = id ? typeById(id) : null;
  openSheet(t ? "Editar tipo de multa" : "Novo tipo de multa", `
    <form id="tf" class="stack" novalidate>
      <label class="f">Categoria<input id="tc" list="cats" value="${esc(t?.catego
      <label class="f">Nome<input id="tn" value="${esc(t?.name || "")}" required>
      <label class="f">Preço unitário (€)<input id="tp" type="number" min="0" ste
      ${t ? `<label class="row"><input type="checkbox" id="tact" ${t.active ? "ch
      <div id="terr" class="err" hidden></div>
      <button class="btn primary block" type="submit">Guardar</button>
    </form>`, () => {
    $("#tf").onsubmit = async (e) => {
      e.preventDefault();
      const row = { category: $("#tc").value.trim(), name: $("#tn").value.trim(),
      if (!row.category || !row.name || $("#tp").value === "" || cents($("#tp").v
      if (t) row.active = $("#tact").checked; else row.sort_order = Math.max(0, .
      const { error } = t ? await sb.from("scrg_fine_types").update(row).eq("id",
      if (error) { $("#terr").textContent = /unique|duplicate/i.test(error.messag
```

```
      closeSheet(); toast("Tipo de multa guardado."); await loadAll(); render();
    };
  });
}

/* ================ AUTENTICAÇÃO ================ */
function authShell(inner) {
  return `<div class="auth"><div class="auth-hero"><img src="${LOGO}" alt="Emblem
    <div class="auth-body"><div class="card stack">${inner}</div></div></div>`;
}
function screenLogin(msg = "") {
  $("#app").innerHTML = authShell(`
    <h3 style="font-size:1.5rem;text-transform:uppercase">Entrar</h3>
    <form id="lf" class="stack" novalidate>
      <label class="f">Email<input id="le" type="email" autocomplete="username" i
      <label class="f">Palavra-passe<input id="lp" type="password" autocomplete="
      <div id="lerr" class="err" ${msg ? "" : "hidden"}>${esc(msg)}</div>
      <button class="btn primary block" type="submit" id="lb">Entrar</button>
    </form>
    <p class="hint" style="margin:0">Acesso apenas por convite das capitãs. Esque
    <button class="btn ghost sm" id="boot">Sou capitã e tenho um código de ativaç
  $("#lf").onsubmit = async (e) => {
    e.preventDefault(); $("#lb").disabled = true;
    const { error } = await sb.auth.signInWithPassword({ email: $("#le").value.tr
    $("#lb").disabled = false;
    if (error) { $("#lerr").textContent = friendlyError(error); $("#lerr").hidden
    boot();
  };
  $("#boot").onclick = screenBootstrap;
}
function screenBootstrap() {
  $("#app").innerHTML = authShell(`
    <h3 style="font-size:1.5rem;text-transform:uppercase">Ativar conta de capitã<
    <form id="bf" class="stack" novalidate>
      <label class="f">O teu email<input id="be" type="email" inputmode="email" r
      <label class="f">Código de ativação<input id="bc" autocomplete="one-time-co
      <div id="berr" class="err" hidden></div>
      <button class="btn primary block" id="bb" type="submit">Ativar</button>
    </form><button class="btn ghost sm" id="back">← Voltar</button>`);
  $("#back").onclick = () => screenLogin();
  $("#bf").onsubmit = async (e) => {
    e.preventDefault(); $("#bb").disabled = true;
    const { data, error } = await sb.functions.invoke("scrg-invite", { body: { ac
    let msg = data?.error; if (error) { try { msg = (await error.context.json()).
    if (msg) { $("#bb").disabled = false; $("#berr").textContent = msg; $("#berr"
    const v = await sb.auth.verifyOtp({ token_hash: data.token_hash, type: data.t
    if (v.error) { $("#bb").disabled = false; $("#berr").textContent = friendlyEr
```

```
    screenSetPassword(true);
  };
}
function screenSetPassword(first) {
  $("#app").innerHTML = authShell(`
    <h3 style="font-size:1.5rem;text-transform:uppercase">${first ? "Bem-vinda!"
    <p class="small muted" style="margin:0">Define a tua palavra-passe para entra
    <form id="pf" class="stack" novalidate>
      <label class="f">Palavra-passe <span class="hint">mínimo 8 caracteres</span
      <label class="f">Repetir palavra-passe<input id="p2" type="password" autoco
      <div id="perr2" class="err" hidden></div>
      <button class="btn primary block" type="submit" id="pb">Guardar e entrar</b
    </form>`);
  $("#pf").onsubmit = async (e) => {
    e.preventDefault();
    const a = $("#p1").value, b = $("#p2").value; const err = $("#perr2");
    if (a.length < 8) { err.textContent = "A palavra-passe tem de ter pelo menos
    if (a !== b) { err.textContent = "As palavras-passe não coincidem."; err.hidd
    $("#pb").disabled = true;
    const { error } = await sb.auth.updateUser({ password: a });
    if (error) { $("#pb").disabled = false; err.textContent = friendlyError(error
    boot();
  };
}
async function handleAccessLink() {
  const p = new URLSearchParams(location.search); const tok = p.get("acesso"); if
  const { data: { session: cur } } = await sb.auth.getSession();
  if (cur && !handleAccessLink._ok) {
    $("#app").innerHTML = authShell(`<h3 style="font-size:1.4rem;text-transform:u
      <p class="small" style="margin:0">Já tens uma sessão aberta neste telemóvel
      <button class="btn primary block" id="lk-go">Continuar com este link</butto
    $("#lk-no").onclick = () => { history.replaceState(null, "", location.pathnam
    $("#lk-go").onclick = () => { handleAccessLink._ok = true; handleAccessLink()
    return true;
  }
  history.replaceState(null, "", location.pathname + location.hash);
  clearCache();
  await sb.auth.signOut({ scope: "local" });
  const { error } = await sb.auth.verifyOtp({ token_hash: tok, type: p.get("t") =
  if (error) { screenLogin("Este link já foi usado ou expirou. Pede um novo link
  screenSetPassword(p.get("t") === "i"); return true;
}
async function boot() {
  const { data: { session } } = await sb.auth.getSession();
  if (!session) return screenLogin();
  S.session = session;
  const cache = readCache(session.user.id);
```

```
  const { data: me, error } = await sb.from("scrg_members").select(MEMBER_COLS).e
  const netFail = error && (/Failed to fetch|NetworkError|network/i.test(error.me
  if (netFail && cache) {
    Object.assign(S, { members: cache.members, allMembers: cache.allMembers || ca
    const m = S.members.find(x => x.user_id === session.user.id); if (!m) return
    S.me = m; S.isCap = !!m.is_captain;
  } else {
    if (netFail) return screenLogin("Sem ligação à internet. Tenta novamente quan
    if (error || !me) { clearCache(); await sb.auth.signOut(); return screenLogin
    S.me = me; S.isCap = !!me.is_captain;
    $("#app").innerHTML = authShell('<div class="empty">A carregar as contas da e
    try { await loadAll(); } catch (e) {
      if (cache) Object.assign(S, { members: cache.members, allMembers: cache.all
      else return screenLogin(friendlyError(e));
    }
  }
  const h = location.hash.slice(1);
  if (h.startsWith("atleta-")) { S.view = "perfil"; S.memberId = h.slice(7); } el
  subscribeRealtime();
  render();
}
sb.auth.onAuthStateChange((ev) => { if (ev === "SIGNED_OUT" && S.me) { S.me = nul
(async () => { if (await handleAccessLink()) return; boot(); })();
```